CC BY SA

Carrera de Especialización en Sistemas Embebidos

Sistemas Operativos de Tiempo Real I

Clase 2: STM32CubeIDE
Basic project on FreeRTOS

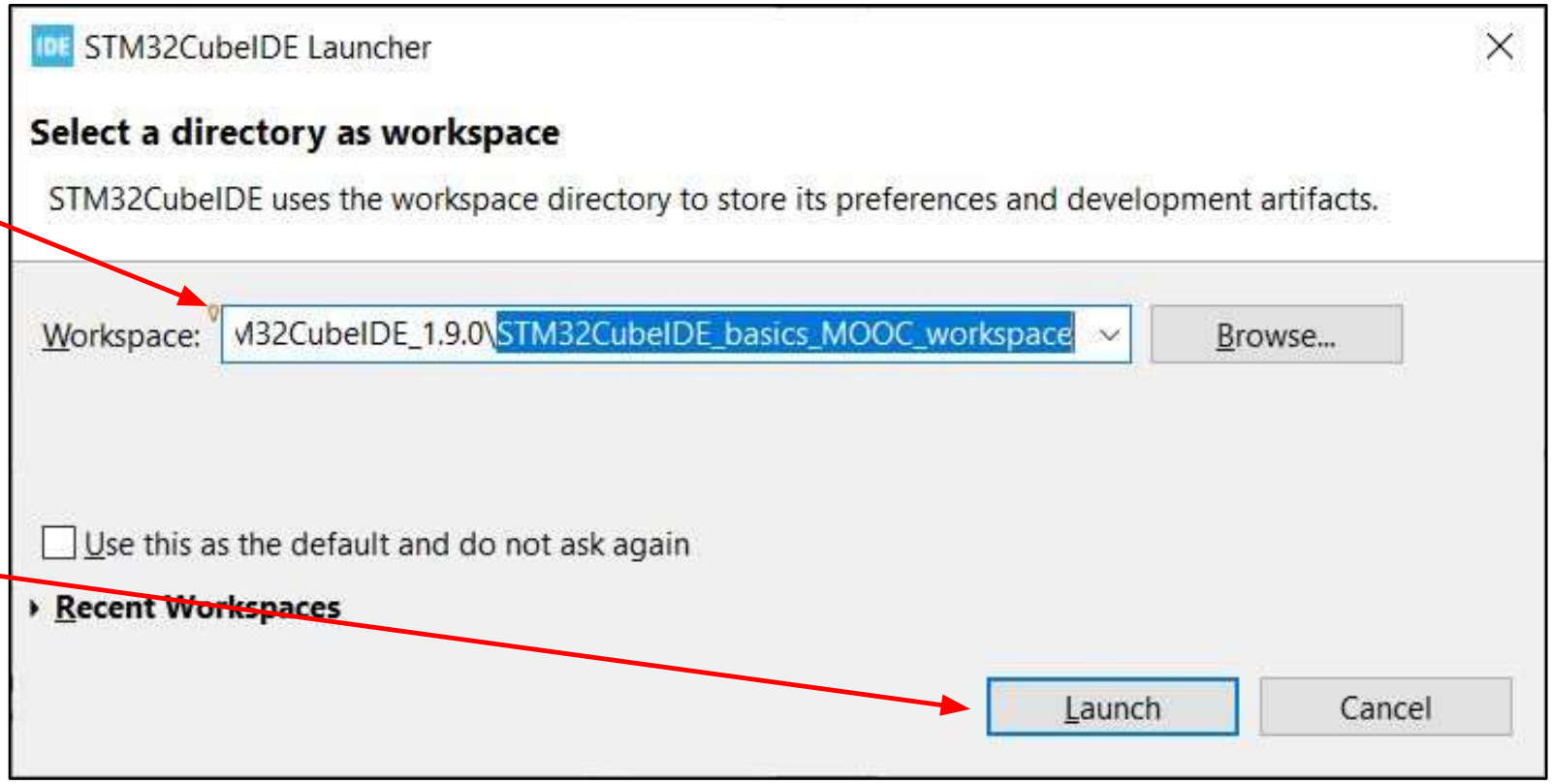
ACSE
Asociación Civil para la Investigación, Promoción y Desarrollo de los Sistemas Electrónicos Embebidos
FACULTAD DE INGENIERIA
Universidad de Buenos Aires
CIM
CADIEEL

Lab: Basic project on FreeRTOS

- Objectives:
  - Create a new project for **NUCLEO-F429ZI** board (STM32-F429ZIT6)
  - Add **FreeRTOS** middleware to the project with CMSIS_OS layer on top
  - Add **task** and one binary **semaphore** and use them to react on **button** press (interrupt) and **LED** control (ON/OFF)

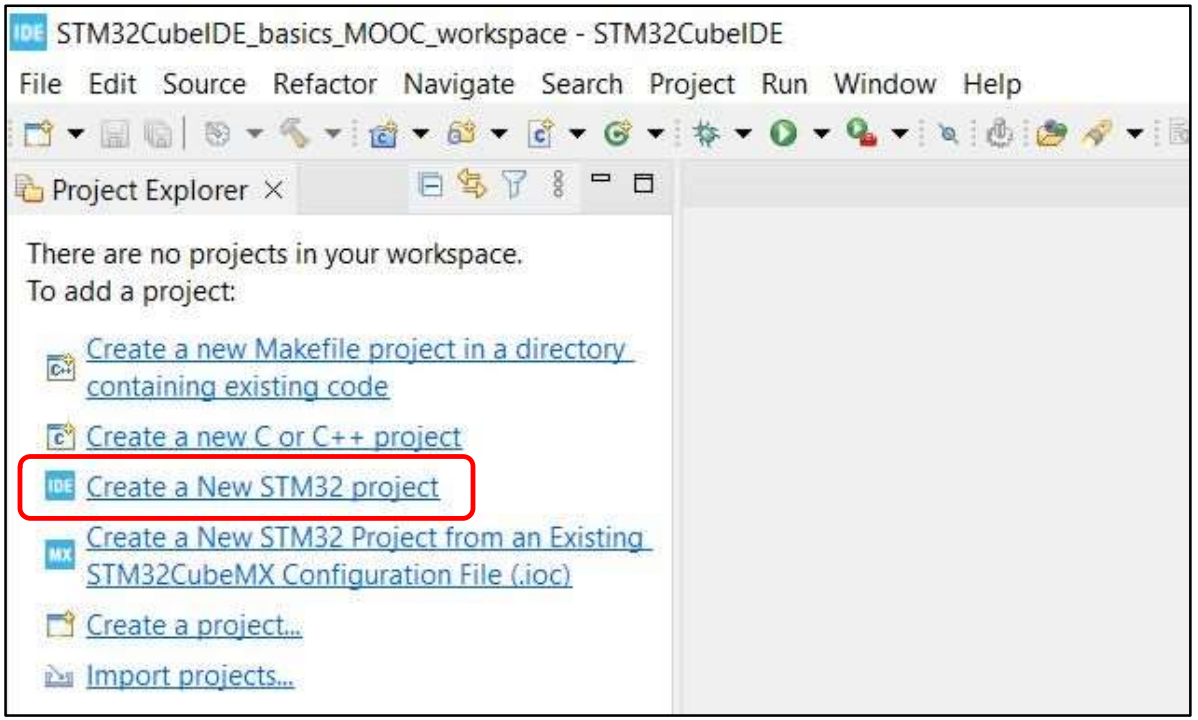
Start a new workspace

- Run **STM32CubeIDE**
- **Select** a folder to store a **workspace**
- Then **Launch**



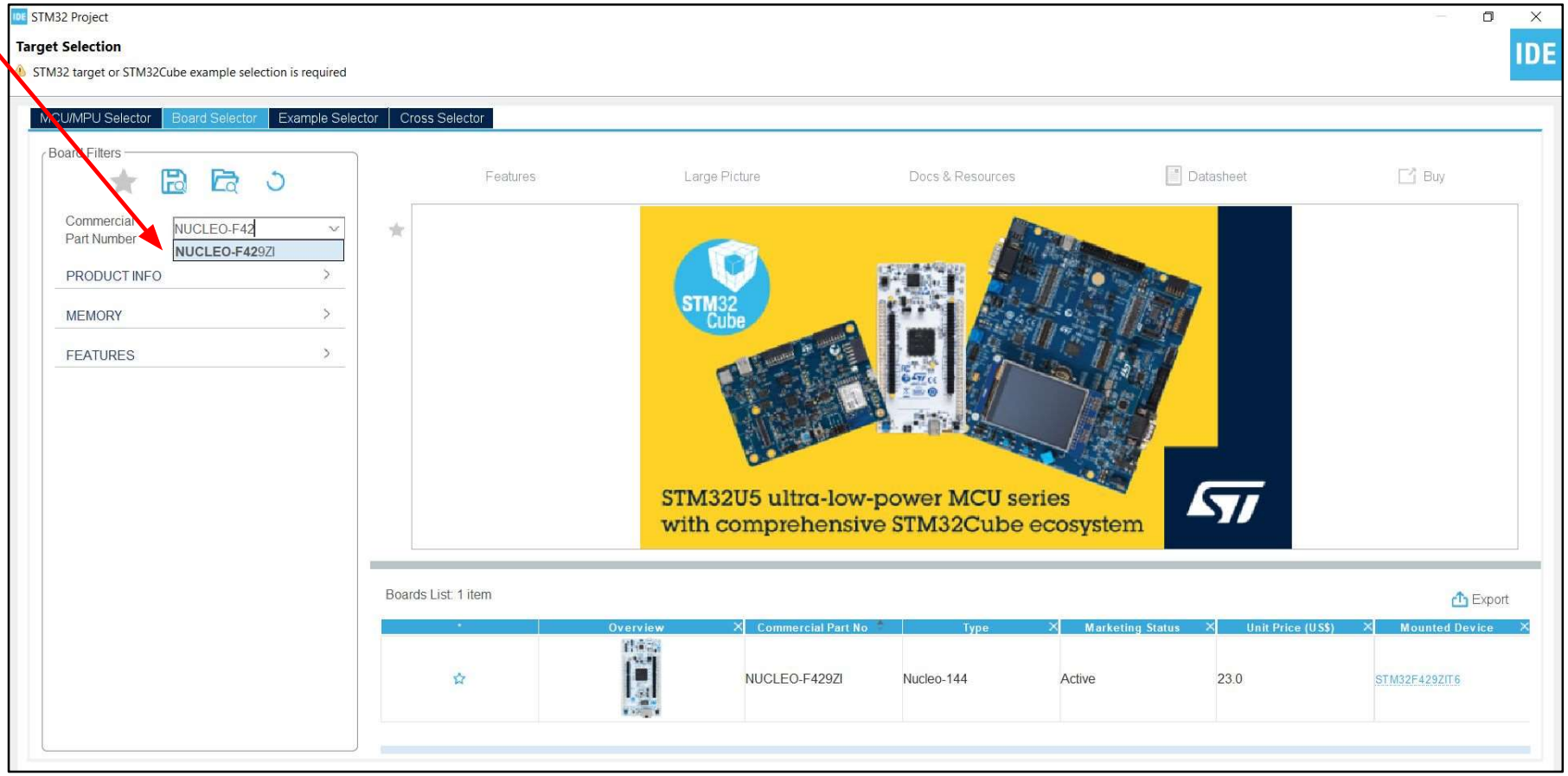
Create a new project

- Click on "**Create a new STM32 project**" to open MCU/Board selector



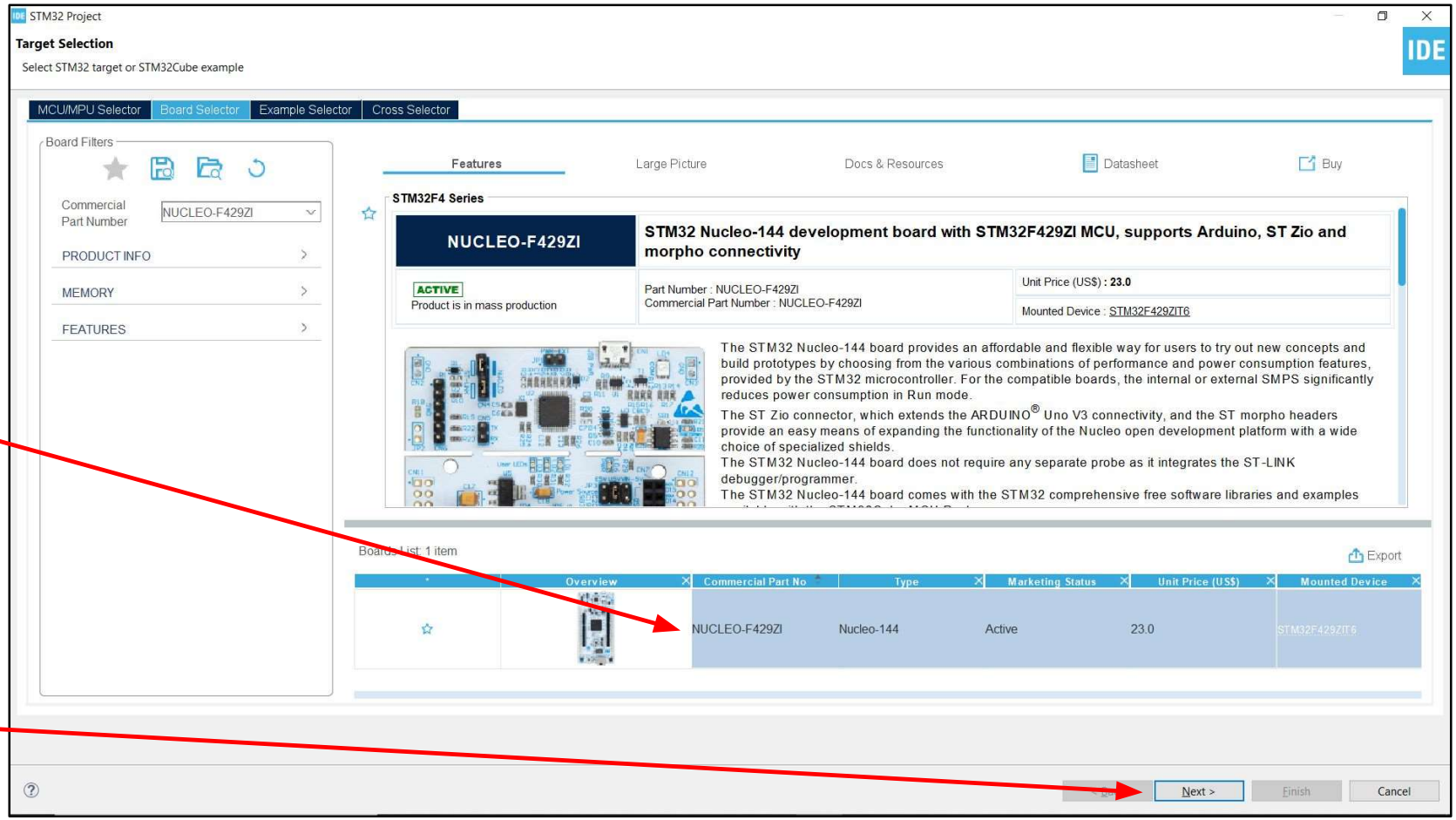
Select target Board: NUCLEO-F429ZI

- We will use **NUCLEO-F429ZI** board (**STM32F429ZIT6** MCU)



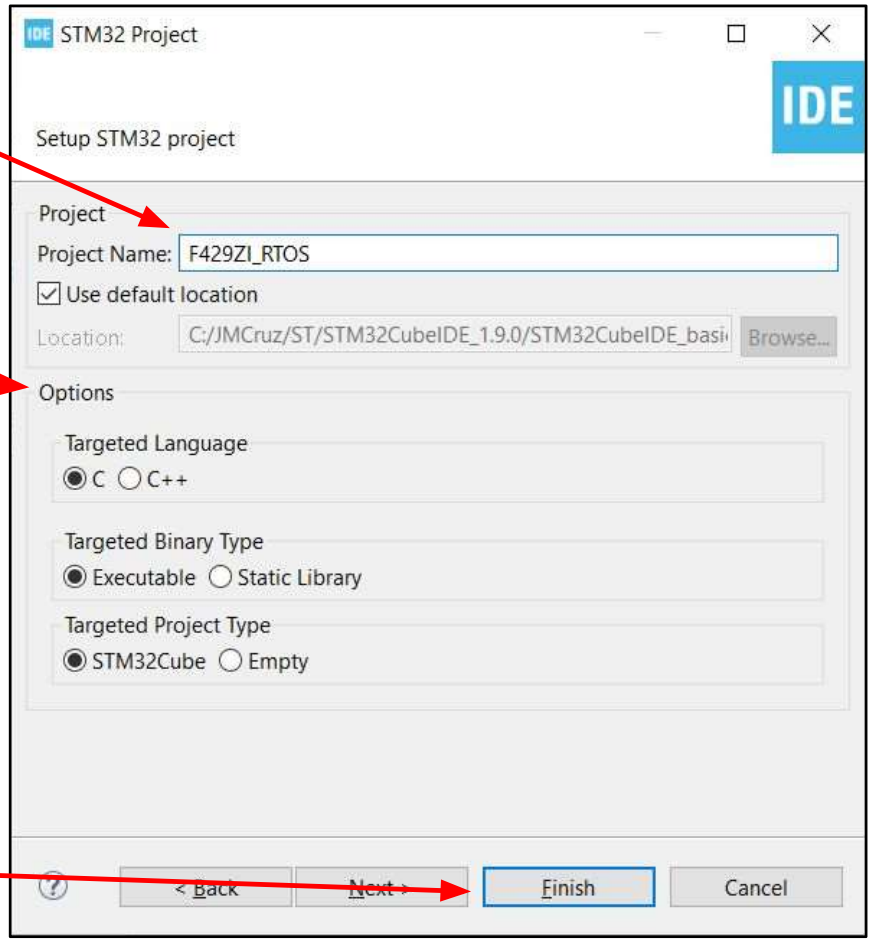
Select target Board: NUCLEO-F429ZI

- It is possible to view on main Board/MCU features, download its documentation
- To start a **new project** we need to double click on the **part number**, then **Next**



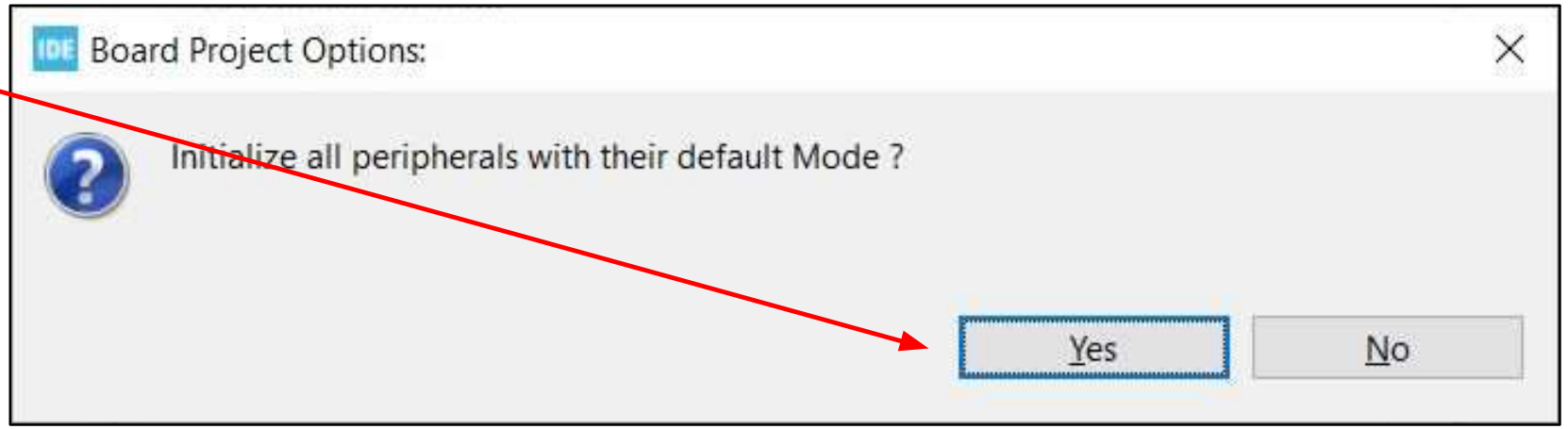
Enter project name

- Specify **project name**, optionally its location (if different from workspace one)
- Additionally we can specify target:
  - language (**C** or C++)
  - binary type (**Executable** or Static Library)
  - project type (generated by **STM32CubeMX** or an Empty one)
- Then **Finish**

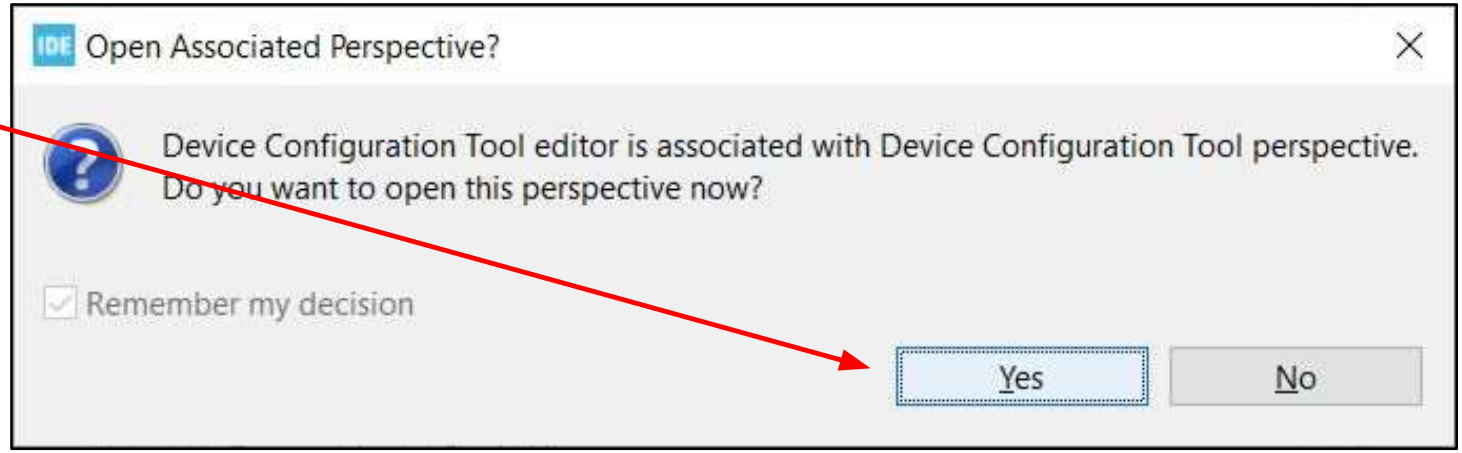


… then and then …

- Then **Yes**

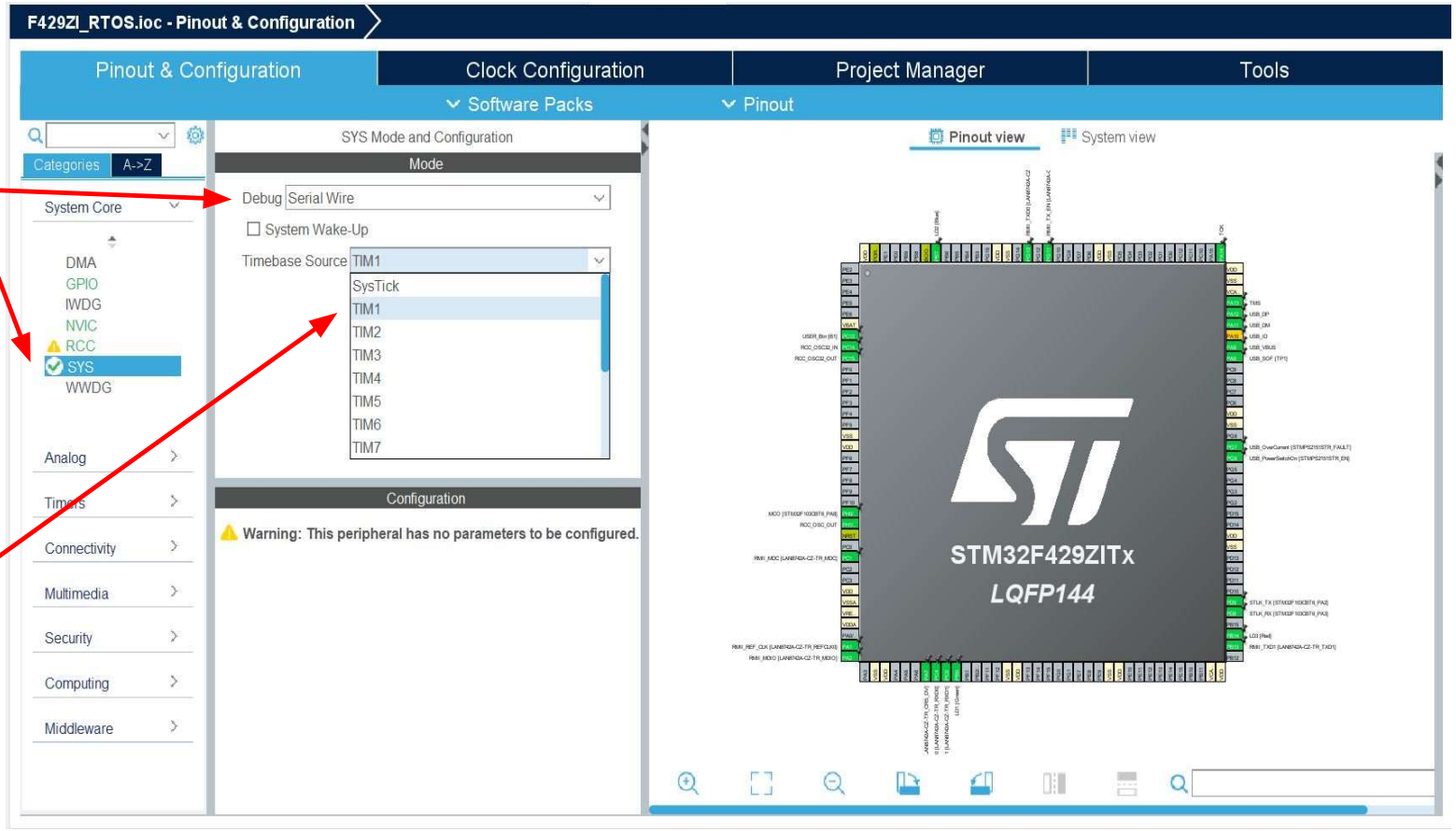


- Then **Yes**



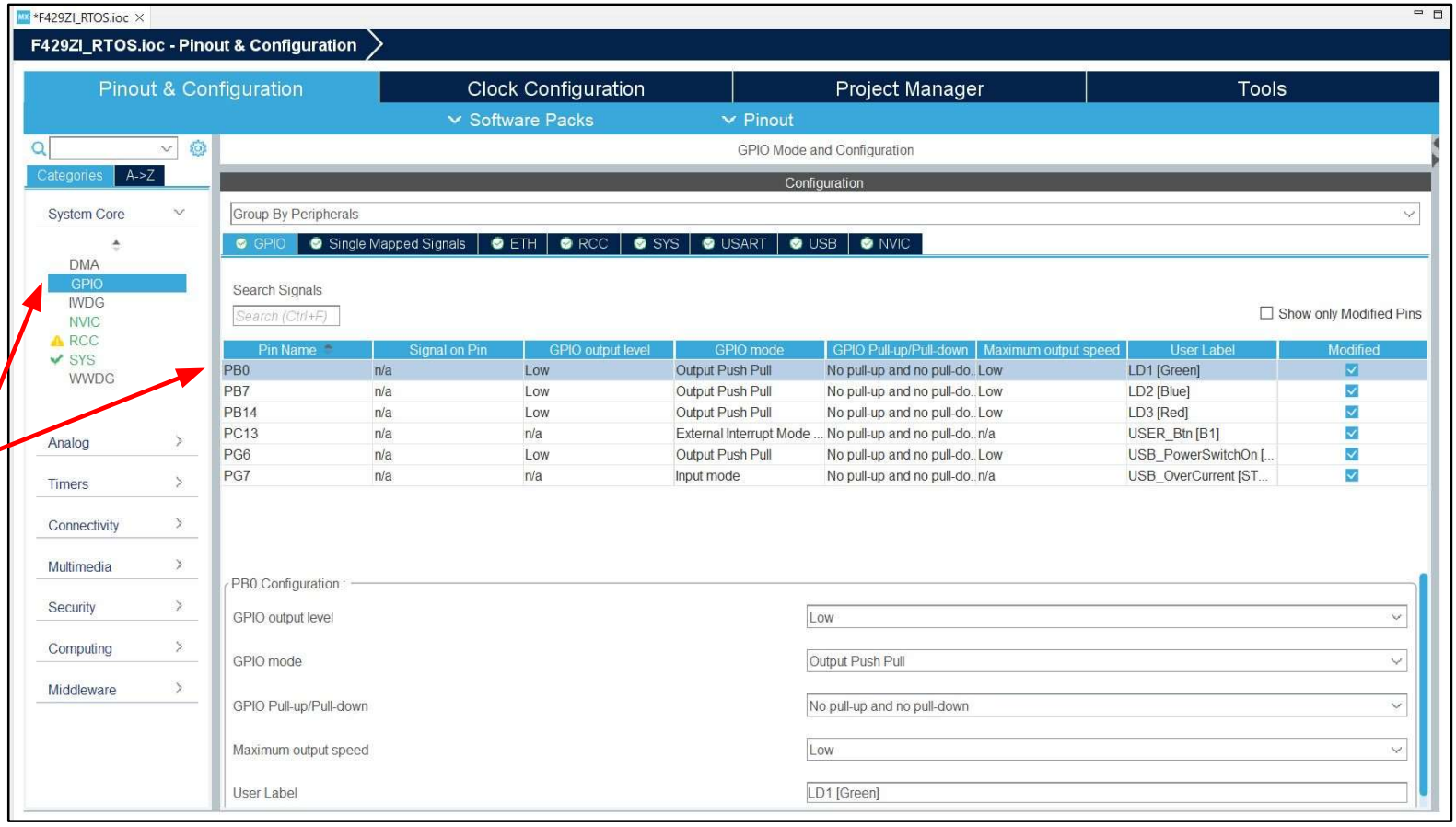
Enabling Serial Wire debug interface

- **Go to** System Core -> **SYS**, SYS tab
- **Check** whether **Serial Wire** is selected
- **Change** a timebase source (used by HAL library functions) from **SysTick** to other timer (i.e. **TIM1**) as SysTick will be used by operating system



Pin Configuration, PB0: LD1 -> [Green]

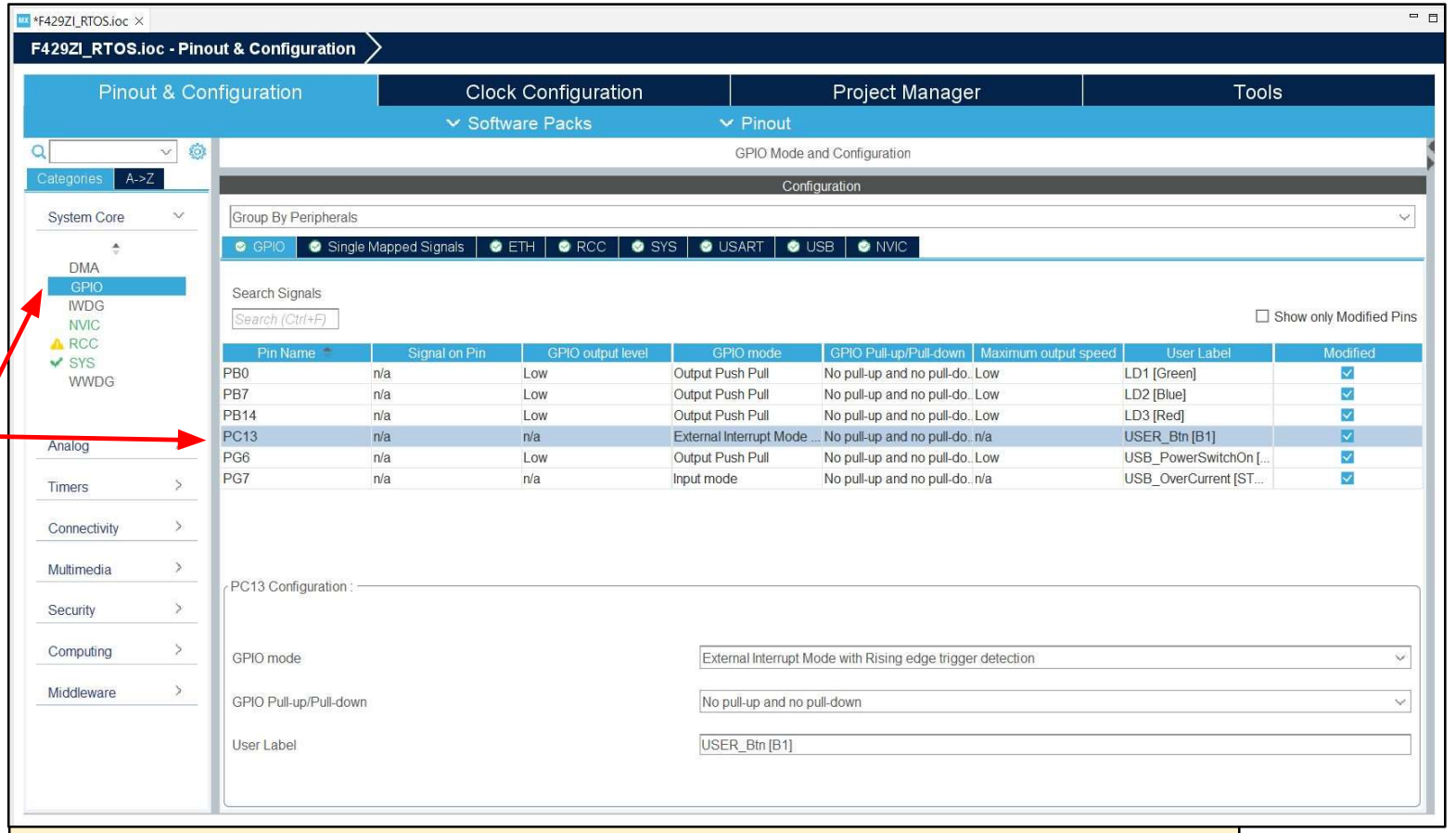
- In this example we are going to use one of the **LEDs** present on the **STM32F429ZIT6** NUCLEO board (**NUCLEO-F429ZI**), connected to **PB0** as seen in from System Core -> **GPIO**, GPIO tab



Hint: Labels are defined in main.h file within generated project (private defines section)

Pin Configuration, PC13: USER_Btn [B1]

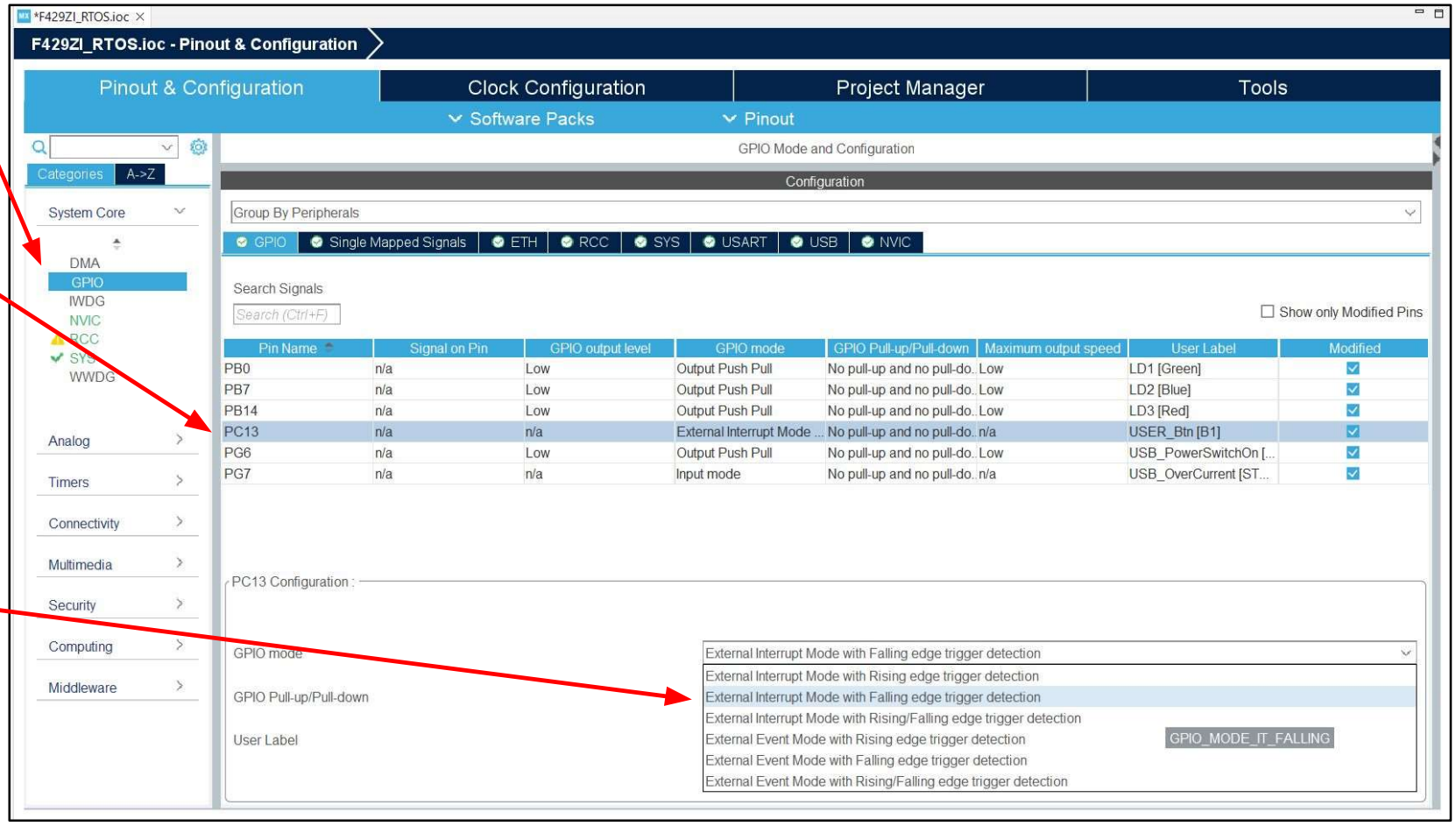
- In this example we are going to use one **button** present on the **STM32F429ZIT6** NUCLEO board (**NUCLEO-F429ZI**) , connected to **PC13** as seen in from System Core -> **GPIO**, GPIO tab



Hint: Labels are defined in main.h file within generated project (private defines section)

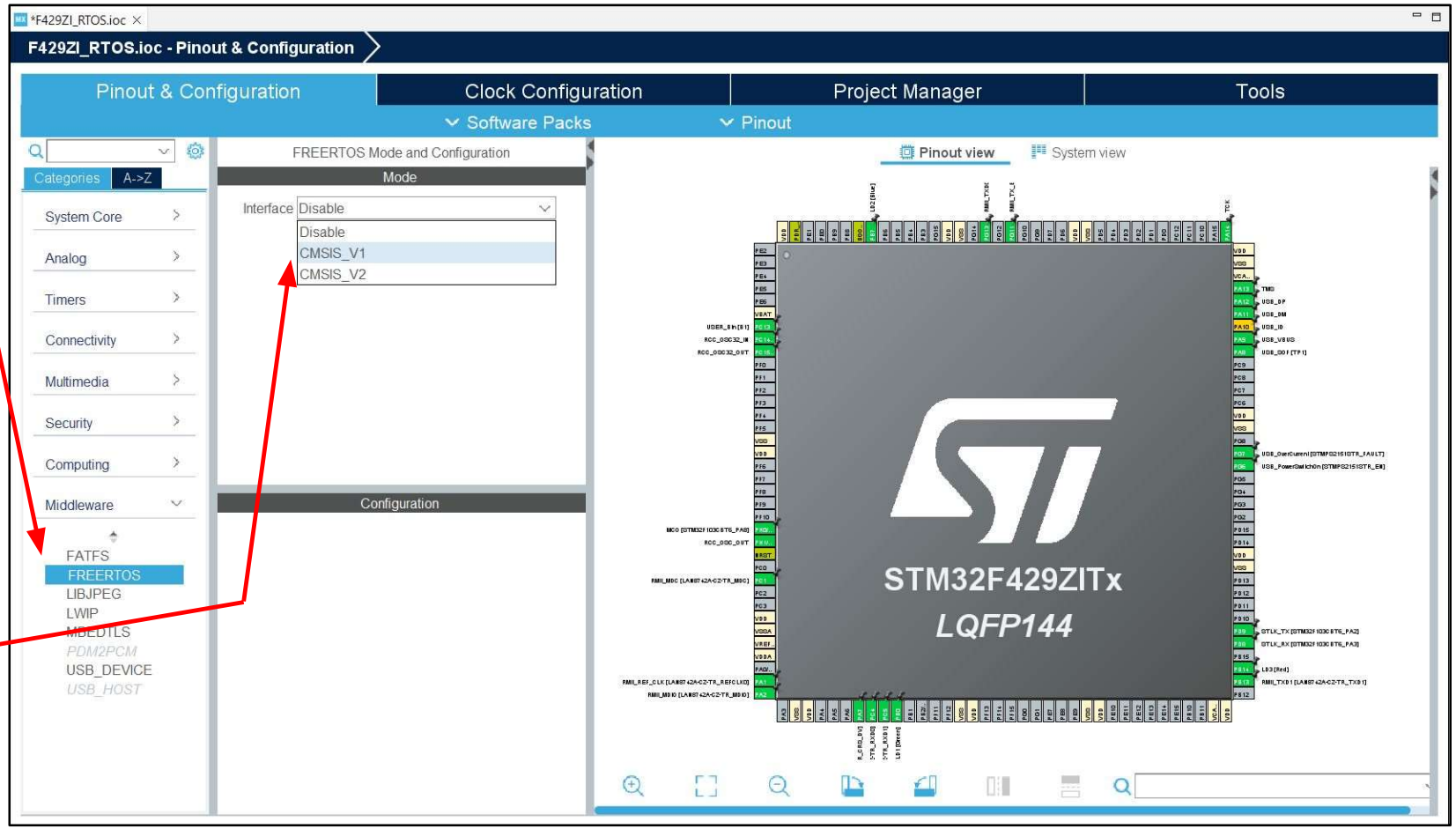
Pin Configuration, PC13: USER_Btn [B1]

- **In to** System Core -> **GPIO**, GPIO tab
- Select **PC13** from the list
- **Change** GPIO mode to External Interrupt Mode with Falling edge trigger detection



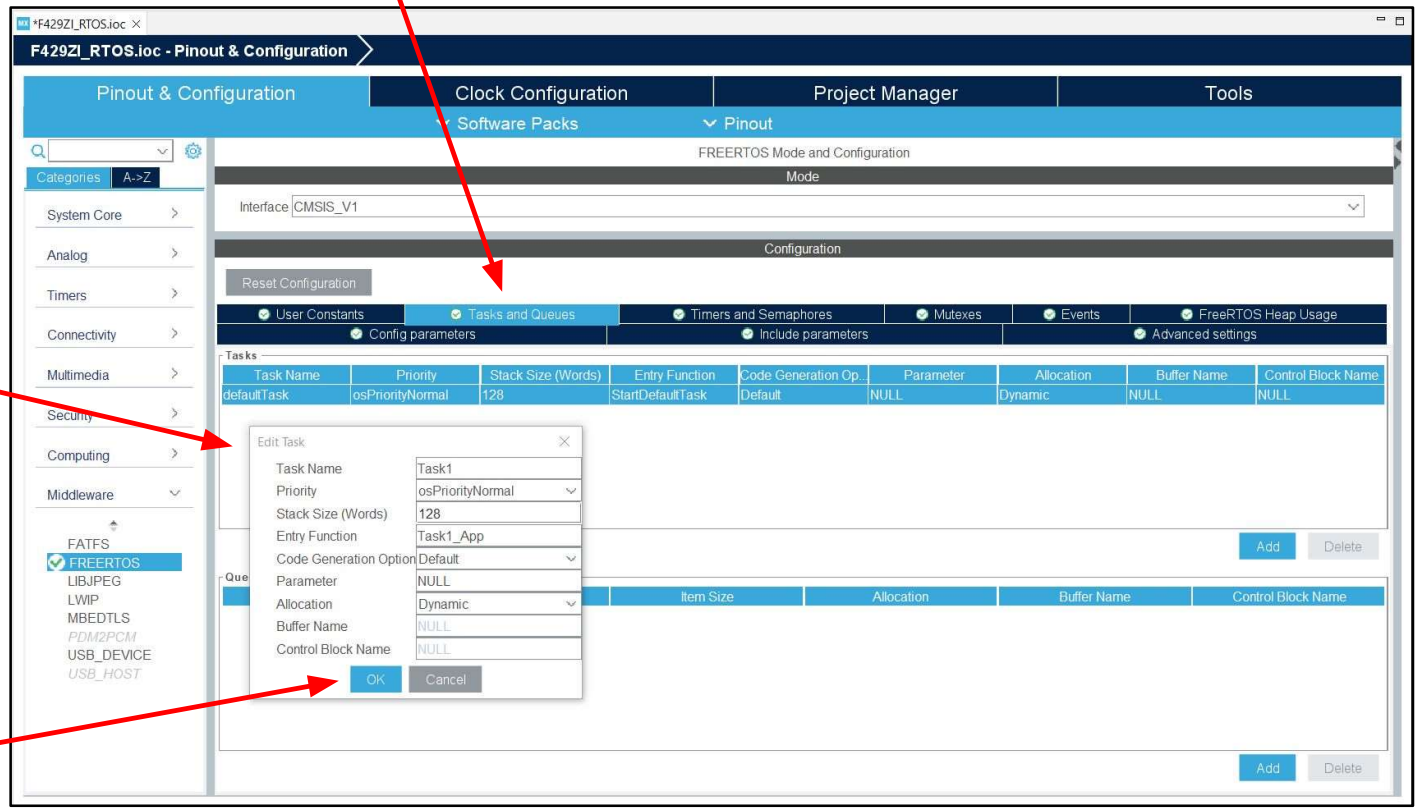
Add FreeRTOS middleware

- Go to Middleware -> **FreeRTOS**, FreeRTOS tab
- **Select** its interface as CMSIS_V1



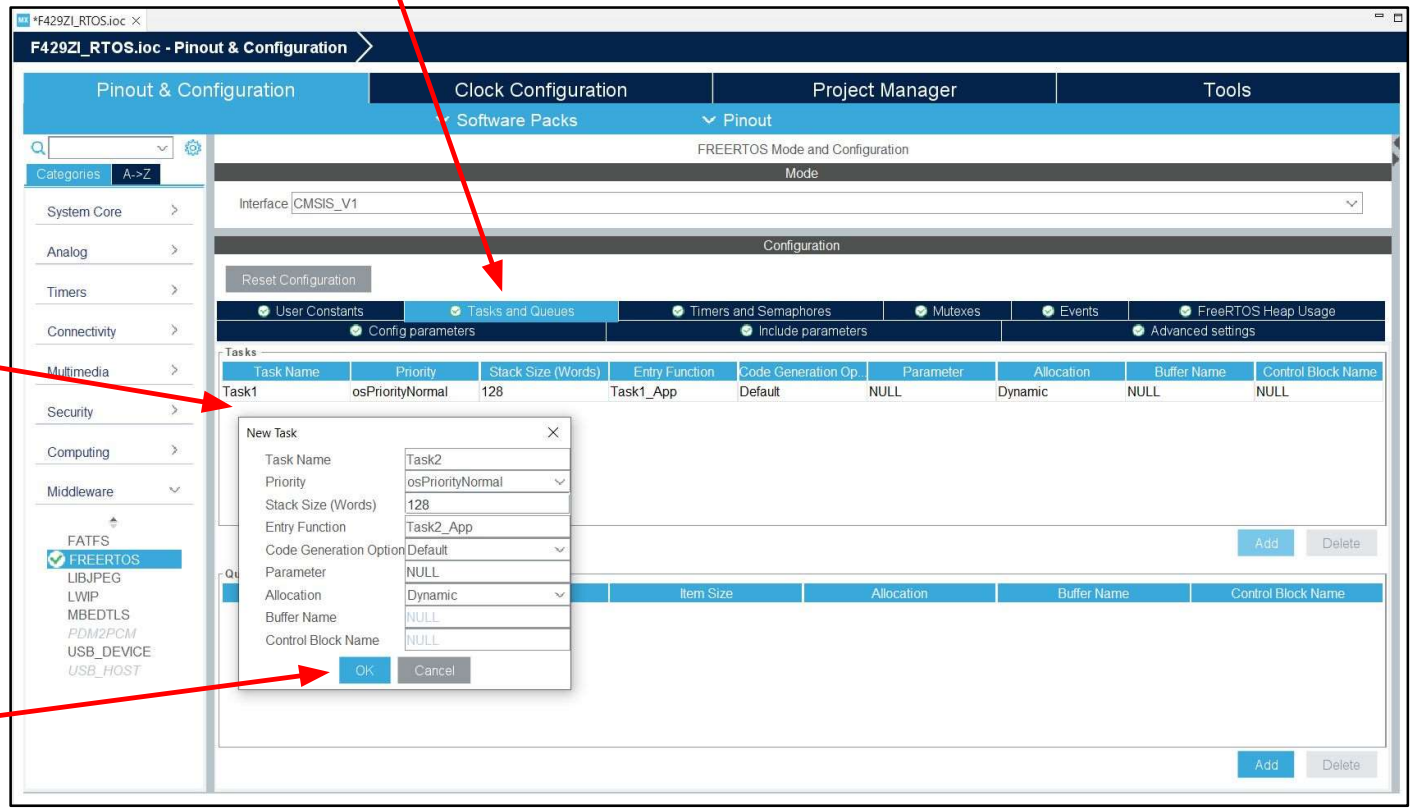
Add two tasks whithing FreeRTOS

- **Go to** FreeRTOS Configuration, **Tasks** and Queues tab
- **Change** default Task to:
  - Task Name: Task1
  - Priority: osPriorityNormal
  - Stack Size: 128 Words
  - Entry Function: Task1_App
- Then **OK**



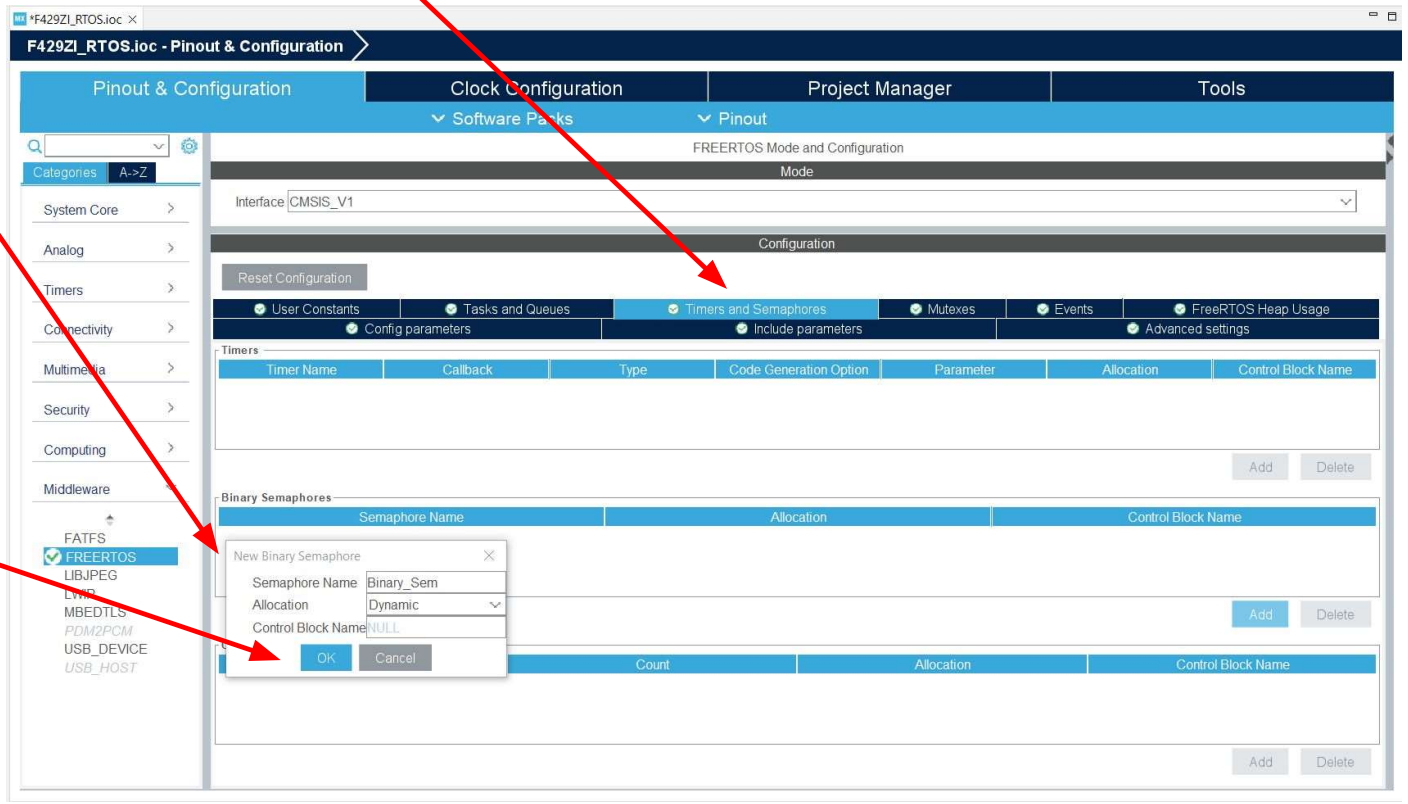
Add two tasks whithing FreeRTOS

- **Go to** FreeRTOS Configuration, **Tasks** and Queues tab
- **Add** new Task:
  - Task Name: Task2
  - Priority: osPriorityNormal
  - Stack Size: 128 Words
  - Entry Function: Task2_App
- Then **OK**



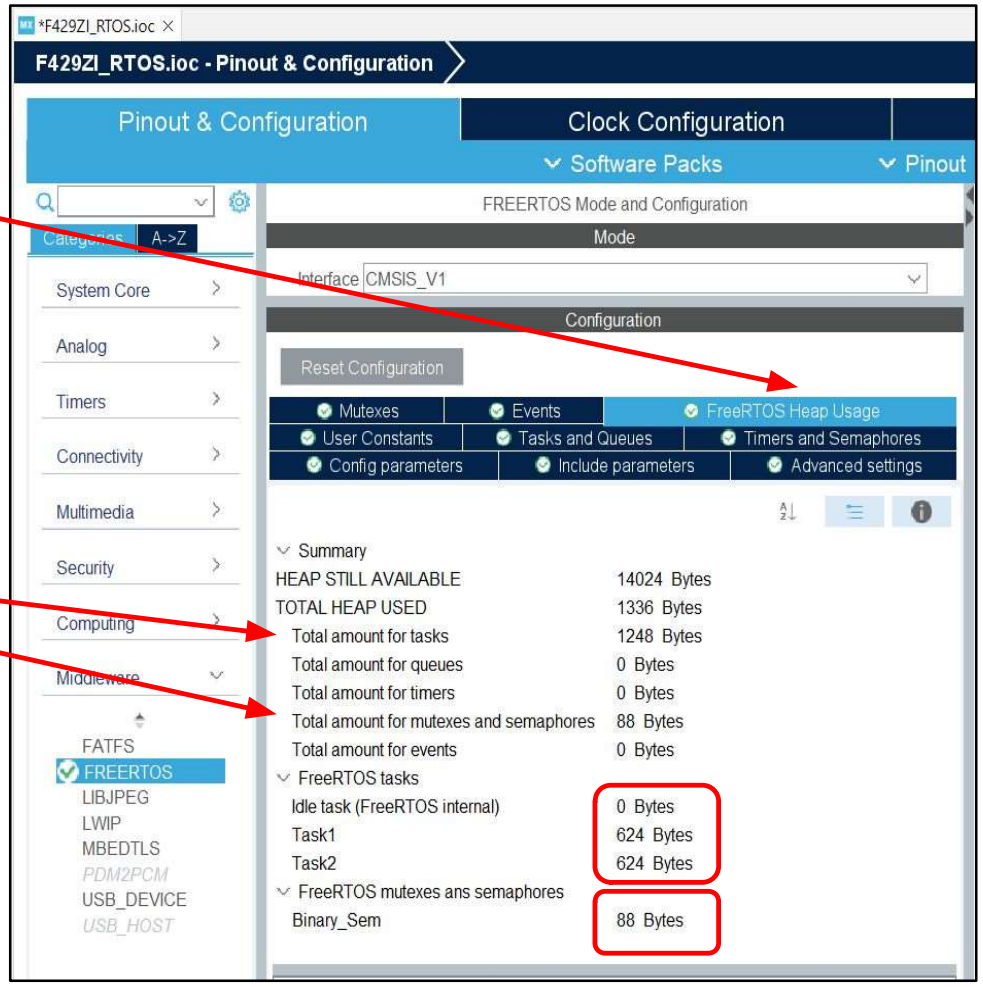
Add binary semaphore within FreeRTOS

- **Go to** FreeRTOS Configuration, Timers and **Semaphores** tab
- **Add** new Binary Semaphore
  - Semaphore Name: Binary_Sem
- Then **OK**



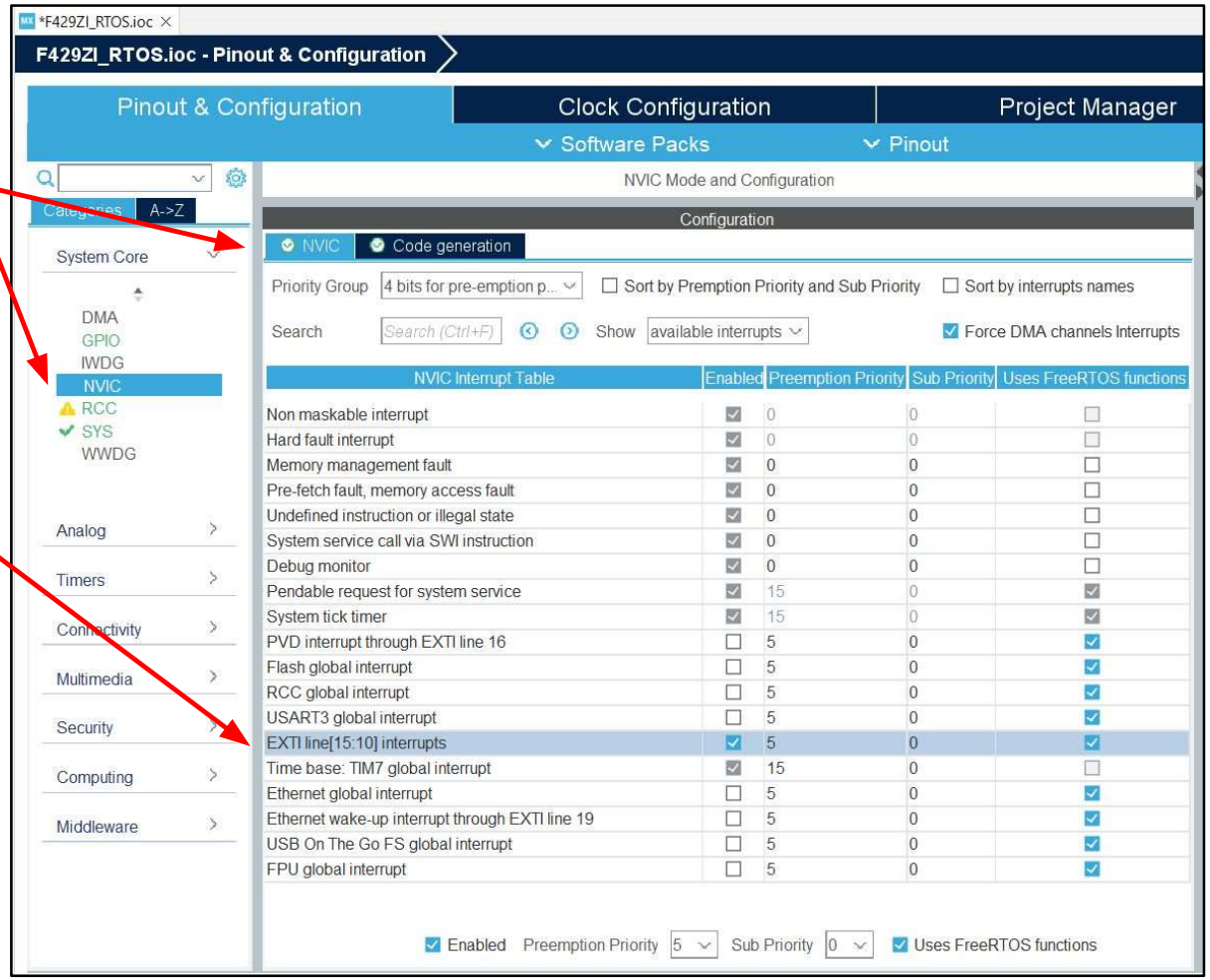
View how much memory OS has used

- **Go to** FreeRTOS Configuration, FreeRTOS **Heap** USAGE tab
- **Check** how many bytes is used by **tasks**, **semaphores**

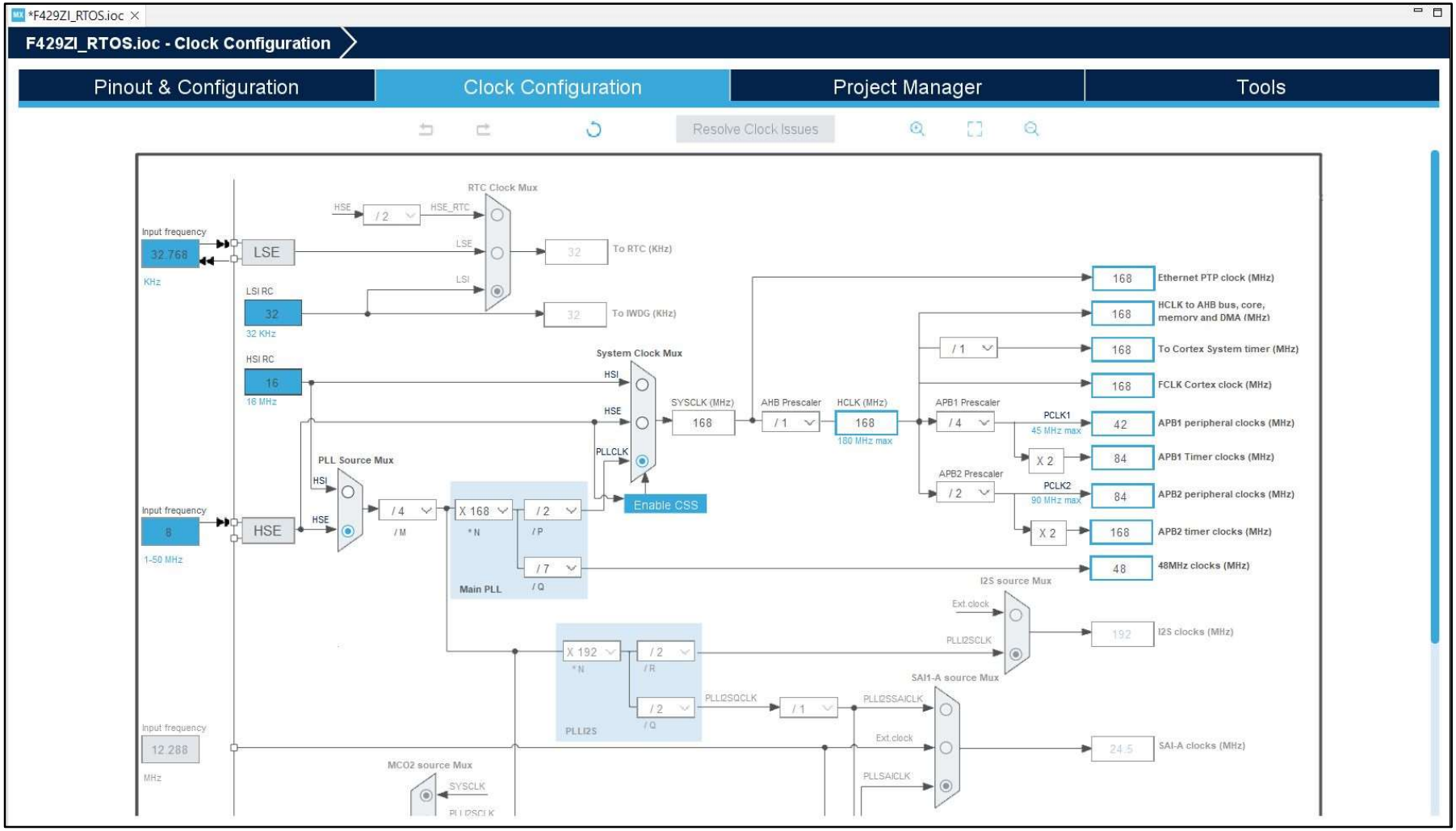


Configure NVIC

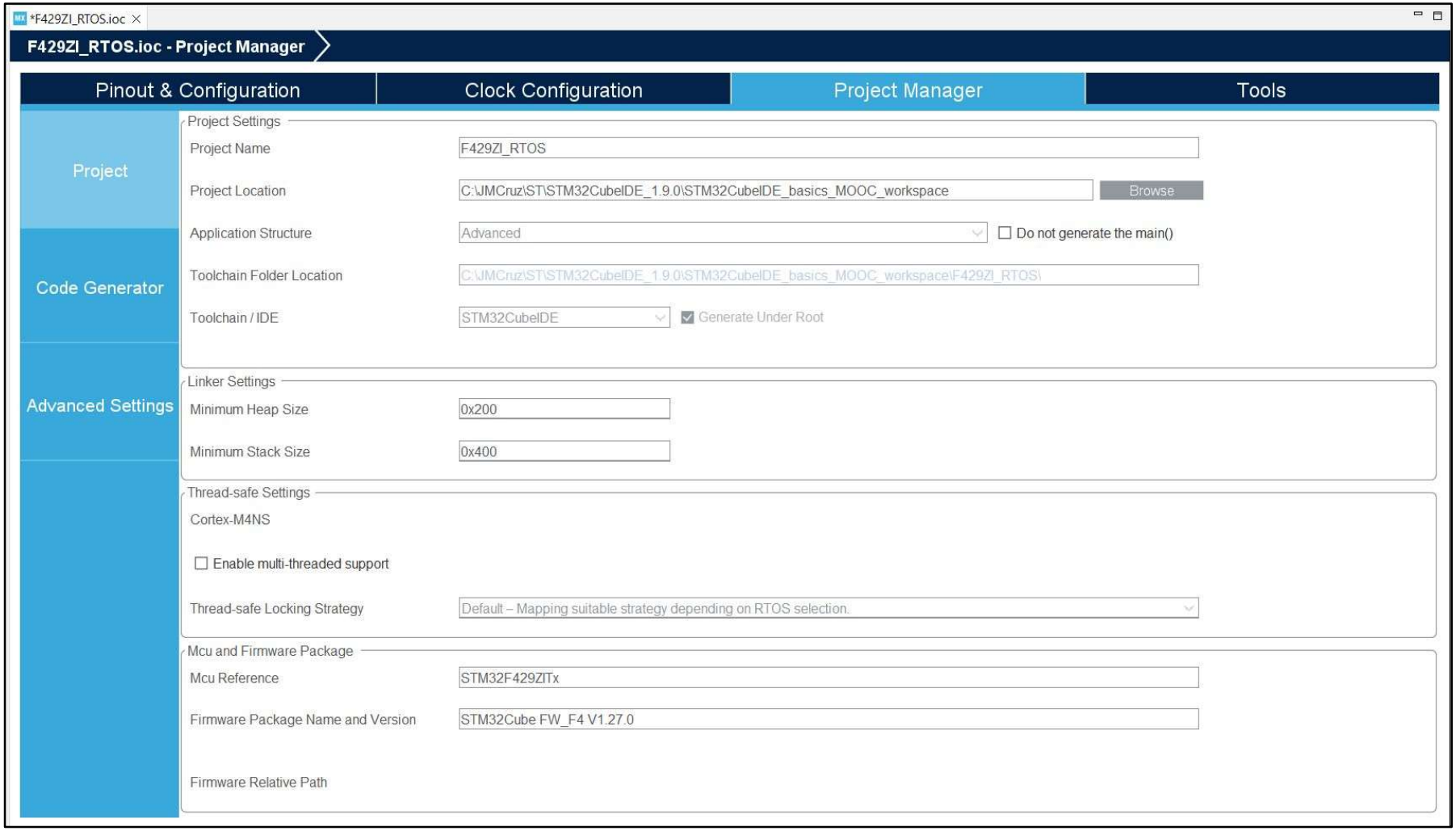
- **Go to** System Core -> **NVIC**, NVIC tab
- **Check** whether EXTI line 15 to 10 interrupts is **marked** as Uses FreeRTOS functions and is enabled



Default clock configuration - no change

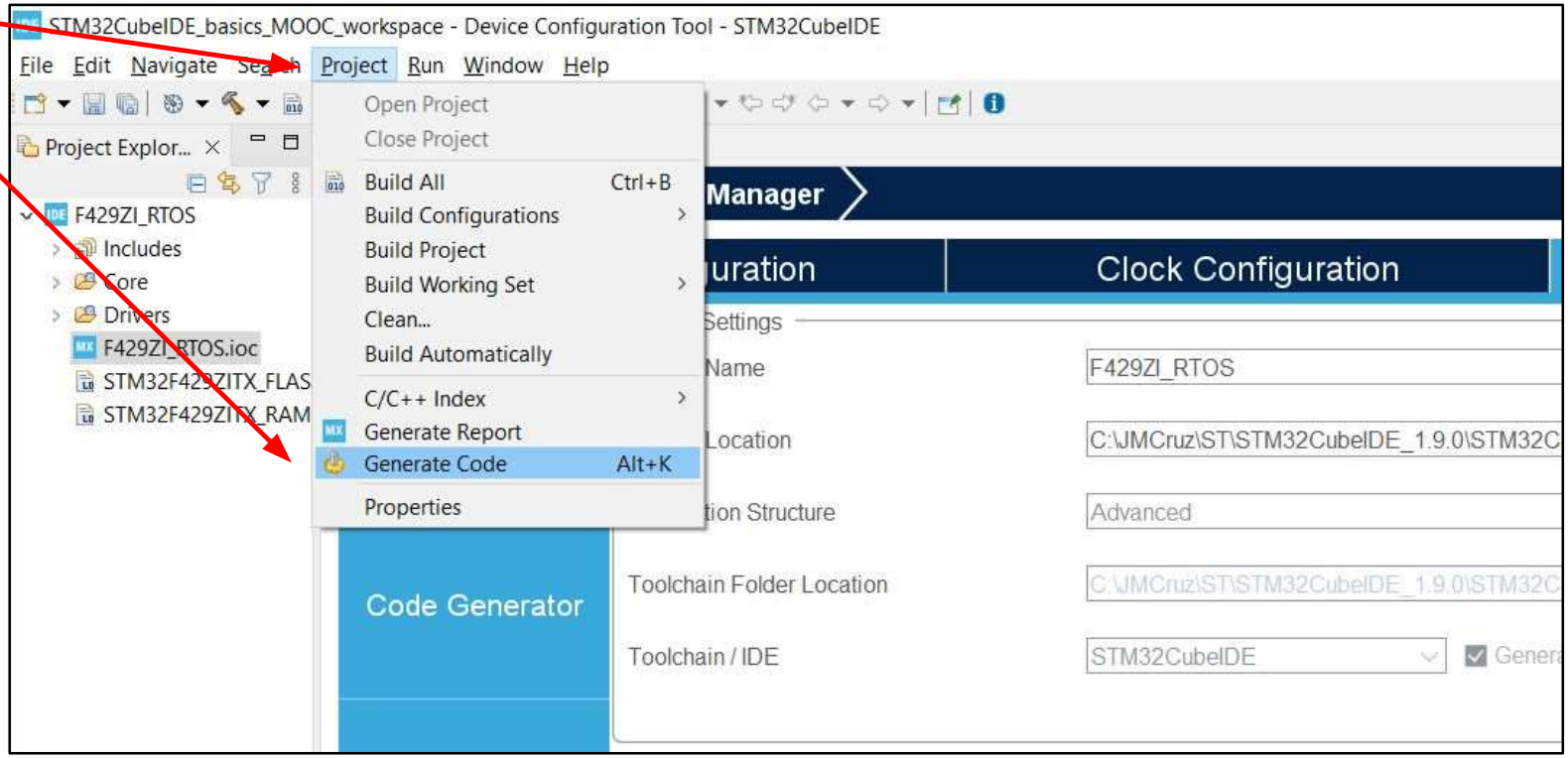


Basic project settings - no change



Code generation

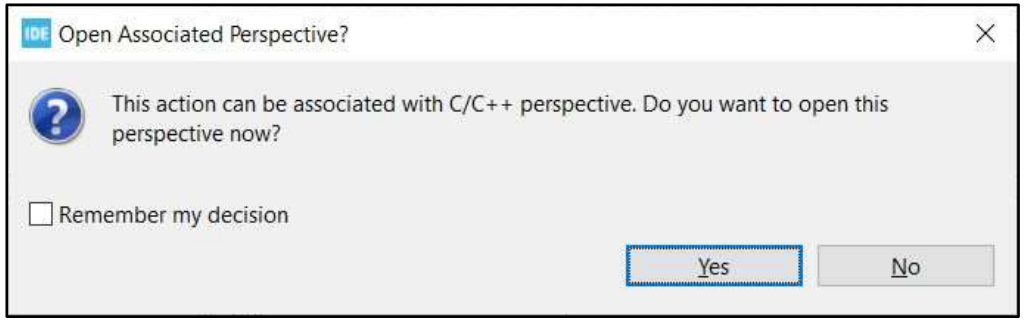
- It is necessary to add to an empty template our project we have just prepared



Hint: It is enough to save the configuration project (.ioc file) to launch project generation

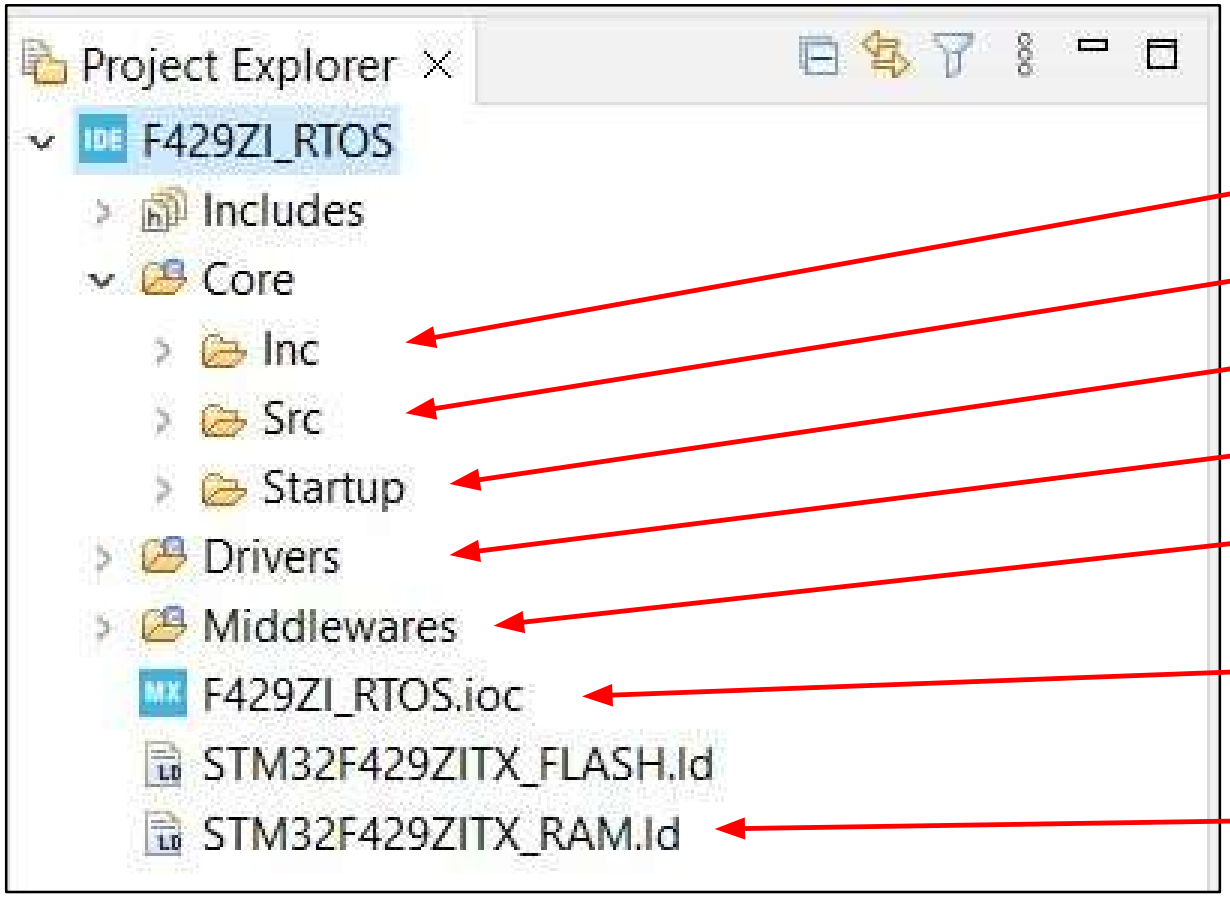
In case we forget about timebase timer

- If we will not change timebase timer from Systick (reserved by FreeRTOS) to any other timer, there would be warning generated before code generation



FreeRTOS base application file structure

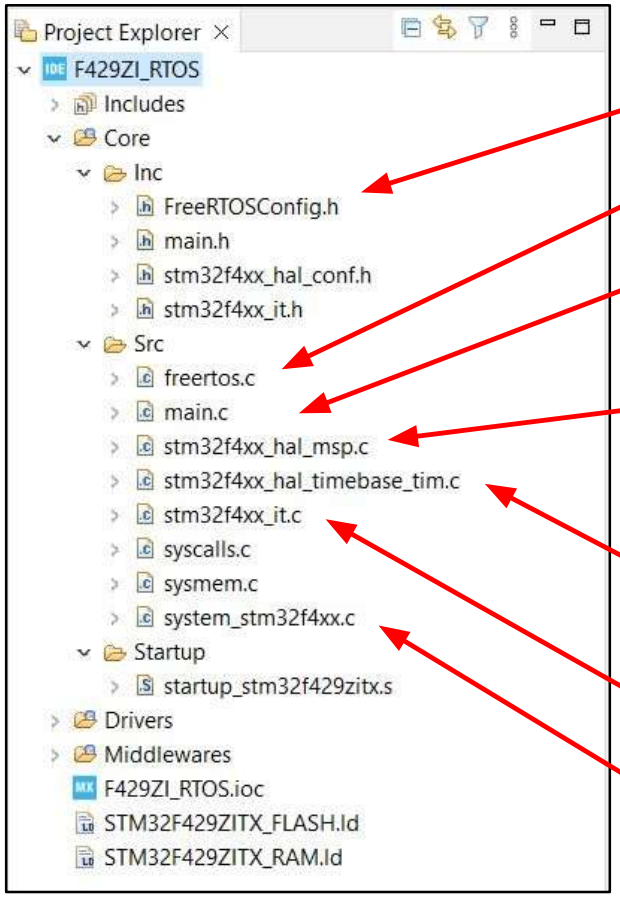
- Once we generate the project, we should see the following file structure



- User application includes
- User application code
- STM32 startup assembly file
- STM32 HAL library
- FreeRTOS code
- Project configuration file (STM32CubeMX style)
- STM32 linker file

User application file structure

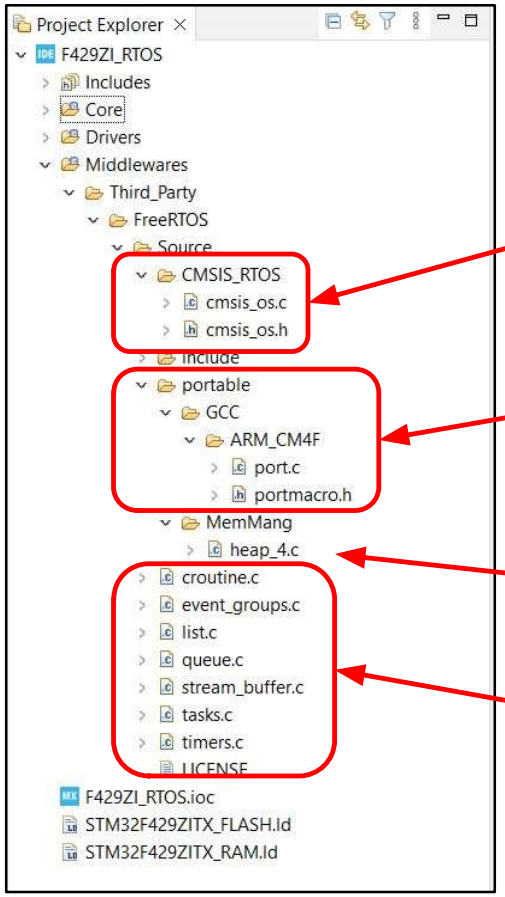
- Once we generate the project, we should see the following file structure



- Main **FreeRTOS** configuration file
- **FreeRTOS** application code (like hooks)
- **Main** user application source file
- **Main** user application HW configuration file (HAL origin)
- **Timebase** routines using selected timer (**TIM7** in our case) used by HAL library for all timing functions
- **Interrupt** routines source file
- Internal **MCU** configuration file (SystemInit() location)

FreeRTOS file structure

- Once we generate the project, we should see the following file structure



- CMSIS_OS layer files - transforming FreeRTOS API functions into CMSIS_OS ones (defined by ARM)
- Interface between FreeRTOS and used MCU (system interrupt vectors definitions)
- Memory management functions to allocate and free the memory resources
- FreeRTOS source files

Code modification: task1 app code - main.c

- **Turn ON** PB0: LD1 -> [Green] within **Task1** application code (**Task1_App**() function), then go to Blocked state for 1 second

```
void Task1_App(void const * argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_SET);
    osSemaphoreWait(Binary_SemHandle, osWaitForever);
  }
  /* USER CODE END 5 */
}
```

Code modification: task2 app code - main.c

- **Turn OFF** PB0: LD1 -> [Green] within **Task2** application code (**Task2_App**() function), then go to Blocked state for 1 second

```
411 void Task2_App(void const * argument)
412 {
413   /* USER CODE BEGIN Task2_App */
414   /* Infinite loop */
415   for(;;)
416   {
417     HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_RESET);
418     osDelay(1);
419   }
420   /* USER CODE END Task2_App */
421 }
```

Application run

- Let´s **compile** the code and **run** the debug session
- As a result we could see either PB0: LD1 -> [Green] always **ON** or **OFF** as both tasks are active for a very short time one just after the other

Code modification: task1 app code - main.c

- **Wait** for Binary_Sem semaphore (being in blocked state)

```
392 void Task1_App(void const * argument)
393 {
394   /* USER CODE BEGIN 5 */
395   /* Infinite loop */
396   for(;;)
397   {
398     HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_SET);
399     osSemaphoreWait(Binary_SemHandle, osWaitForever);
400   }
401   /* USER CODE END 5 */
402 }
```

Code modification: BLUE_BUTTON interrupts callback

- Release Binary_Sem on each BLUE_BUTTON press

```
381 /* USER CODE BEGIN 4 */
382 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
383 {
384   if(USER_Btn_Pin == GPIO_Pin)
385     osSemaphoreRelease(Binary_SemHandle);
386 }
387 /* USER CODE END 4 */
```

Application run

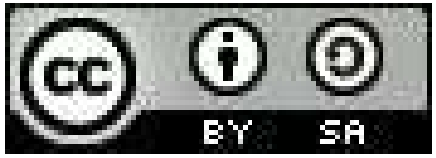
- Let´s **compile** the code and **run** the debug session
- As a result on each blue button press we are unblocking **Task1** (PB0: LD1 -> [Green] **turn on**), then it depends on which stage is **Task2** (PB0: LD1 -> [Green] **turn off**) for how long our LED will be in ON state

Referencias

- STM32CubeIDE basics MOOC: https://www.st.com/content/st_com/en/support/learning/stm32-education/stm32-moocs/STM32CubeIDE_basics_MOOC.html
- STMCubeIDE basics - CMSIS_OS lab: Basic project on FreeRTOS: https://drive.google.com/file/d/1qEyOZMpbUxyxspFoYr1ymg74gwz9ZTh6/view?usp=sharing

CC BY SA

Licencia



STM32CubeIDE Basic project on FreeRTOS

Por Ing. Juan Manuel Cruz, se distribuye bajo una

licencia de Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional